18-19 Final

1 a) One of the major drawbacks of traditional file processing system is the *difficulty to access data*. Explain it with a suitable example. Mention some widely used applications of modern database systems. [10]

b) A major purpose of a database system is to provide users with an abstract view of the data. Explain the concept of different types of *data abstraction* in modern database systems. [10]

c) What are projection and selection? Does the order of these operations matter in the final result? Explain with example. [05]

Sol

A) Drawbacks of Traditional File Processing System and Modern Database Applications:

Traditional File Processing System:

In traditional file processing systems, data is stored in separate files and accessed through application programs. Retrieving specific information often involves complex and time-consuming processes. For example, consider a library system where books are stored in text files. To find the details of a specific book, the application would have to scan through the entire file, leading to slow and inefficient data retrieval.

Modern Database Applications:

Modern databases, on the other hand, offer efficient data access and management. Some widely used applications include:

1. **Online Transaction Processing (OLTP) Systems:** Used in banking, e-commerce, and retail for real-time transaction processing.
2. **Customer Relationship Management (CRM) Systems:** Manage customer information, interactions, and transactions.
3. **Enterprise Resource Planning (ERP) Systems:** Integrate various business processes like finance, human resources, and inventory management.
4. **Data Warehousing and Business Intelligence Systems:** Analyze large datasets to derive insights for decision-making.
5. **Content Management Systems (CMS):** Manage digital content for websites, blogs, and other online platforms.

B) Data Abstraction in Modern Database Systems:

Data Abstraction:

Data abstraction in modern database systems refers to presenting complex data in a simplified, organized, and user-friendly manner. There are three types of data abstraction:

- **Physical Data Independence:** Shielding users from the physical storage details of data. Users are not concerned with where and how data is stored physically.
- **Logical Data Independence:** Shielding users from changes in the logical structure of the data. Users can modify the schema without affecting the application programs.
- **View Data Independence:** Allowing different users to have different views (subsets) of the data. Users can see only the relevant portions of the data.

C) Projection and Selection in Database Operations:

Projection: Projection in the database context involves selecting specific columns from a table while excluding others. It reduces the number of columns in the result set.

Selection: Selection involves filtering rows from a table based on specific criteria using conditions in the `WHERE` clause. It reduces the number of rows in the result set.

The order of these operations can impact the final results. For example, consider a table `Employees` with columns `EmployeeID`, `Name`, `Department`, and `Salary`.

1. **Selection before Projection:**

```sql
SELECT Name, Department
FROM Employees
WHERE Salary > 50000;
```

In this case, rows where `Salary > 50000` are selected first, and then only `Name` and `Department` columns are projected.

2. **Projection before Selection:**

```sql
SELECT Name, Department
FROM Employees
```

Here, all rows are selected first, and then only `Name` and `Department` columns are projected. Later, a separate selection operation can be applied if needed.

The order matters because selection affects the number of rows, while projection affects the number of columns. Performing selection first can reduce the number of rows before unnecessary columns are included in the result set, potentially improving query performance.

2(A+B partial) → Prev. Yr

B → 2nd Part ERD → SQL Quiz

2(C) → Inner JOIN vs OUTERJOIN, → Prev. Yr + Slide

3(A)

Following are the requirements of some tables. Each field is separated by comma (,) and additional requirements are stated in brackets [ ]. [10]

- Departments(ID primary key, Name of dept [Can not be empty], Establishment Year)
- Employees(ID primary key [it is exactly a 9-digit number without any decimal part], Name [20 characters long but it can be in any language other than English],Date of birth [must not be empty and it can be less than 01-01-1960], Dept [foreign key of departments entity and it can not be empty], Blood Group [must be any one from A+,A-, B+,B-,AB+,AB-]
- Salary(EmpID [foreign key of Employees entity], Date of Payment, Amount)

The tasks are:

i. Create the tables using standard SQL.

ii. Now you forgot to include Salary (per month) and Join Date of the Employees. How can you solve this problem?

i

1. Creating Tables (Original Requirements):

sql — Copy code

```sql
CREATE TABLE Departments (
    ID INT PRIMARY KEY,
    Name VARCHAR(255) NOT NULL,
    EstablishmentYear INT
);

CREATE TABLE Employees (
    ID Number(9,0),
    Name VARCHAR(30) NOT NULL,
    DateOfBirth DATE NOT NULL,
    Dept varchar(10),
    BloodGroup CHAR(3),
    FOREIGN KEY (Dept) REFERENCES Departments(ID)
);

CREATE TABLE Salary (
    EmpID INT,
    DateOfPayment DATE,
    Amount DECIMAL(10, 2),
    FOREIGN KEY (EmpID) REFERENCES Employees(ID)
);
```

ii

2. Adding Missing Columns (Salary and Join Date):

If you forgot to include `Salary` (per month) and `JoinDate` of the `Employees` table, you can solve this problem by altering the `Employees` table and adding the missing columns. Here's how you can modify the `Employees` table to include these columns:

sql — Copy code

```sql
ALTER TABLE Employees
ADD Salary DECIMAL(10, 2),
ADD JoinDate DATE;
```

This `ALTER TABLE` statement adds the `Salary` column of type `DECIMAL(10, 2)` (representing currency with 2 decimal places) and the `JoinDate` column of type `DATE` to the `Employees` table.

Now, the `Employees` table includes the missing columns `Salary` (representing monthly salary) and `JoinDate` (representing the date an employee joined the company).

b

(b) Write the SQL for the following queries:

i. Find out the name and date of birth of all Employees who joined in the last 3 [illegible] 90 days).

ii. Find out the name, date of birth and name of the department of all Employees [illegible] in the last 3 months (i.e. 90 days).

iii. Generate a list containing department name and its total employees.

iv. Generate a list containing department name and its total employees but include [illegible] partment with at least 20 employees.

v. Generate a list containing department name and its total employees but include [illegible] partment with at least 20 employees and whose departmental average salary is [illegible] 20000.

1. Find out the name and date of birth of all Employees who joined in the last 3 months (90 days):

sql — Copy code

```sql
SELECT Name, DateOfBirth
FROM Employees
WHERE JoinDate >= SYSDATE - 90;
```

ii. Find out the name, date of birth, and name of the department of all Employees who joined in the last 3 months (90 days):

sql — Copy code

```sql
SELECT e.Name, e.DateOfBirth, d.Name AS Department
FROM Employees e
JOIN Departments d ON e.Dept = d.ID
WHERE e.JoinDate >= SYSDATE - 90;
```

iii. Generate a list containing department name and its total employees:

```sql
SELECT d.Name AS Department, COUNT(e.ID) AS TotalEmployee
FROM Departments d
LEFT JOIN Employees e ON d.ID = e.Dept
GROUP BY d.Name;
```

iv. Generate a list containing department name and its total employees but include departments with at least 20 employees:

```sql
SELECT d.Name AS Department, COUNT(e.ID) AS TotalEmployee
FROM Departments d
LEFT JOIN Employees e ON d.ID = e.Dept
GROUP BY d.Name
HAVING COUNT(e.ID) >= 20;
```

v. Generate a list containing department name and its total employees but include the departments with at least 20 employees and whose departmental salary is greater than 20000:

```sql
SELECT d.Name AS Department, COUNT(e.ID) AS TotalEmployee
FROM Departments d
LEFT JOIN Employees e ON d.ID = e.Dept
WHERE e.Salary > 20000
GROUP BY d.Name
HAVING COUNT(e.ID) >= 20;
```

a

In the context of databases, **cardinality** refers to the relationship between two tables, specifically the number of unique values that can exist in a column of a table. It defines the uniqueness of data values in a particular column and helps establish relationships between tables. There are three main types of cardinality:

1. One-to-One (1:1) Cardinality:

In a one-to-one relationship, each record in the first table corresponds to exactly one record in the second table, and vice versa. For example, consider two tables: `Person` and `Passport`. Each person can have only one passport, and each passport belongs to only one person. This is a rare relationship type due to the limited scenarios where one-to-one relationships are necessary.

2. One-to-Many (1:N) Cardinality:

In a one-to-many relationship, a record in the first table can correspond to multiple records in the second table, but a record in the second table can only correspond to one record in the first table. For example, consider two tables: `Customer` and `Orders`. One customer can place multiple orders, but each order is placed by only one customer. This is a common relationship type in database design.

3. Many-to-Many (N:N) Cardinality:

In a many-to-many relationship, records in both tables can correspond to multiple records in the other table. For example, consider two tables: `Students` and `Courses`. A student can enroll in multiple courses, and each course can have multiple students. To represent this type of relationship in a relational database, a junction table is used. This junction table contains foreign keys from both tables, creating two one-to-many relationships.

Understanding cardinality is crucial for designing efficient and normalized databases, ensuring accurate data relationships between entities in complex systems.

4(a) → Prev. Yr

(c) Integrity constraint

In a relational database, there are four main types of integrity constraints that help maintain data accuracy and consistency. Each constraint ensures specific rules are followed when performing operations on the database. Here are the four integrity constraints with suitable cardinalities:

1. Primary Key Constraint:

- **Cardinality:** One-to-One (1:1)
- **Description:** Ensures that a column or a set of columns in a table uniquely identifies each record in that table. No two rows can have the same primary key value. It establishes a one-to-one relationship between the primary key column(s) and the corresponding data rows.

2. Foreign Key Constraint:

- **Cardinality:** One-to-Many (1:N)
- **Description:** Establishes a link between two tables, where the foreign key column in one table refers to the primary key column in another table. Each value in the foreign key column of the child table corresponds to a unique value in the primary key column of the parent table. This enforces a one-to-many relationship.

3. Unique Constraint:

- **Cardinality:** One-to-One (1:1)
- **Description:** Ensures that the values in a column or a set of columns are unique across all the rows in the table. Unlike the primary key constraint, unique constraints allow null values but ensure that non-null values are unique. Each unique constraint enforces a one-to-one relationship for the unique column(s).

4. Check Constraint:

- **Cardinality:** One-to-Many (1:N)
- **Description:** Specifies a condition that the values in a column must satisfy. It restricts the range of allowed values in the column. While it can enforce a one-to-one relationship for specific values, it can also be used to establish a one-to-many relationship if the condition allows multiple values that meet the criteria.

These integrity constraints are fundamental for maintaining data integrity and ensuring accurate relationships within a relational database.

5(a) What is domain of atribute. Explain different type of atribute used in ER model.

⇒

In the context of databases and the Entity-Relationship (ER) model, an **attribute domain** refers to the set of all possible values that an attribute can have. It defines the range of valid values that a specific attribute can store. The domain of an attribute helps in understanding the type and constraints on the data that can be stored in that attribute.

In the ER model, attributes can be of various types. Here are the different types of attributes used in the ER model:

1. Simple Attribute:

A simple attribute is an atomic, indivisible attribute that cannot be further divided. It represents a single value for an entity. For example, the attribute "Age" for an entity "Person" is a simple attribute.

2. Composite Attribute:

A composite attribute is an attribute that can be divided into smaller sub-parts, each with its own meaning. Composite attributes are made up of multiple simple attributes. For example, the attribute "Address" can be composite and consist of sub-attributes like "Street," "City," and "Zip Code."

3. Derived Attribute:

A derived attribute is an attribute whose value can be derived or calculated from other attributes in the database. Derived attributes are not physically stored in the database but are computed when needed. For example, "Age" can be a derived attribute if it is calculated based on the "Date of Birth" attribute.

4. Multi-valued Attribute:

A multi-valued attribute can have multiple values for a single entity. For example, the attribute "Phone Numbers" for an entity "Person" can be multi-valued if a person can have multiple phone numbers.

5. Key Attribute:

A key attribute is an attribute that is used as a unique identifier for an entity within an entity set. It uniquely identifies an entity within its entity set. For example, the "EmployeeID" attribute can be a key attribute for the "Employee" entity.

6. Single-valued Attribute:

A single-valued attribute can have only a single value for a specific entity. Most simple attributes fall into this category, where each entity has one specific value for that attribute.

Understanding these attribute types is essential for designing a well-structured and normalized database schema in the ER model, ensuring that data is accurately represented and relationships are properly defined.